

Merge Intervals

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example Test Cases:

TEST CASE 1

Input: Intervals = $[[1,3],[2,6],[8,10],[15,18]]$

Output: $[[1,6],[8,10],[15,18]]$

Explanation: Since intervals $[1,3]$ and $[2,6]$ overlap, merge them into $[1,6]$.

TEST CASE 2

Input: Intervals = $[[439, 539],[539,541]]$

Output: $[[439, 541]]$

Explanation: Intervals $[439, 539]$ and $[539,541]$ are considered overlapping.

TEST CASE 3

Input: Intervals = $[[4,7],[1,4]]$

Output: $[[1,7]]$

Explanation: Intervals $[1,4]$ and $[4,7]$ are considered overlapping.

Expected Time Complexity: $O(n \log n)$

Input Format:

An integer 2D array `intervals` of size $n \times 2$, where each interval is represented as

$$\text{intervals}[i] = [\text{start}_i, \text{end}_i]$$

with $\text{start}_i \leq \text{end}_i$.

Output Format:

Return a 2D array containing the merged non-overlapping intervals such that all overlapping intervals in `intervals` are combined, and the resulting intervals cover the same ranges as the input. Return the merged non-overlapping intervals, listed from left to right on the number line (i.e., in increasing order of their starting points).

Assumptions:

1. $1 \leq t \leq 1000$
2. $1 \leq n \leq 2 \cdot 10^5$
3. Each interval is of the form $[\text{start}_i, \text{end}_i]$ where $\text{start}_i \leq \text{end}_i$.
4. start_i and end_i lie in the range $[-10^9, 10^9]$.
5. It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

(After reading this question, follow the steps described in the `Instructions.pdf` file to see the exact input and output format for the provided test cases.)

Instructions:

You have been provided with two template files, containing the `main` and `solve` functions respectively. The main function parses the input for each testcase. For each sub-test-case, you have to call the `solve` function, which computes and returns the required sum (the maximum possible sum of the array after performing the aforementioned operation any number of times).

The `solve` function must compute this sum in $O(n \log n)$ time.

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)

Library Usage

For this lab, you are permitted to use any modules which are part of the standard library of either C++ or python.